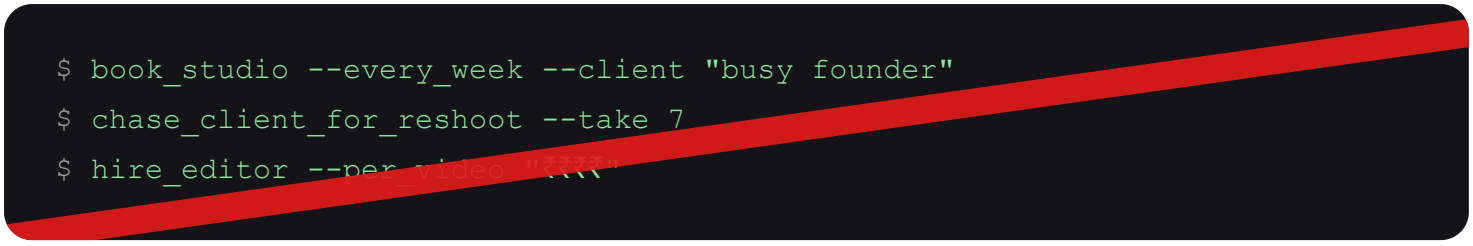
FREE GUIDE · @AUTOMATEWITHANAND

The AI Avatar Agency
3 Tools. Clients Never Film.

the exact system I run at ZeroHands — one filming session per client, then video forever



None of that happens in my agency. A client films **once** — one short session with a videographer — and from that day, every video is built by three tools: **HeyGen** (their face), **ElevenLabs** (their timing), and **Claude Code** (the brain that runs everything). This guide is the full breakdown I promised: what each tool does, the exact setup, the copy-paste prompts and files, and how I run it for clients so they never pick up a camera again.

The system on one page

1. **HeyGen** — train a library of avatar "looks" from one filming session. Type a script, get the client on camera.
2. **ElevenLabs Scribe** — word-level timestamps for every render (~$0.013/reel). This is what makes captions and cuts sync *without an editor*.
3. **Claude Code** — saved skill files write the scripts in the client's voice, pick the edit, and drive the whole build (ffmpeg compositing) from one command.

```
client films ONCE (one session, multiple outfits/angles)
  → HeyGen trains the avatar library
  → per video: Claude writes script → HeyGen renders → Scribe timestamps
  → Claude composites captions + graphics (ffmpeg) → posted
  → repeat weekly. no camera. no editor. no reshoot.
```

---

1 HeyGen — the avatar library

TOOL · THE CLIENT'S FACE, ON DEMAND

HeyGen trains a talking avatar from roughly **2 minutes of real footage**. You type a script, it returns a video of the person saying it — same face, same voice, same set. That's the whole unlock: filming becomes a one-time event, not a weekly chore.

But one avatar isn't enough. If every reel comes from the same clip, the feed screams "one recording session." So at the filming session you capture **multiple looks**: different outfits, different corners of the room, front and side angles. From my own session I trained **17 looks across 5 outfits** — a warm bookshelf set for framework videos, a dark wall with red accents for dramatic hooks, a bright window for tutorial walkthroughs, seated armchair looks for longer talks. Rotating them makes the account look like it films constantly.

Setup, step by step:

- Film ~2 minutes per look: subject talks naturally to camera, hands visible, normal gestures, no cuts.
- Upload each clip in HeyGen → create an avatar look from it. Name them by outfit + setting (`green_bookshelf_front`, `white_darkwall_side`) — you'll thank yourself later.
- Clone the voice in the same session (HeyGen prompts you for a consent line — the client reads it on camera).
- Render from the dashboard, or via API once you're doing volume. The API call is one POST with `avatar_id` + your script text + `voice_id`, then you poll until the MP4 is ready.

My renders run through one command that wraps the API:

```
# list the trained looks and what each is good for
python3.11 reel.py looks

# render a script on a specific look (--yes confirms the credit spend)
python3.11 reel.py render green_bookshelf_front --script-file script.txt --yes
```

Cost ballpark: HeyGen's creator plans start around the price of a lunch per month; on API plans think roughly **one credit ≈ one minute of avatar video**. A weekly reel schedule for a client burns a handful of credits a month — trivially cheap next to a videographer per video. Check current pricing before you quote a client.

Pitfall 1 — every look frames the face differently. A seated armchair look and a standing window look put the head in completely different places. Before you build graphics or crops on top of a new look, render one test, pull a frame, and verify the framing. Skipping this is how you ship a video with the client's forehead cut off.

Pitfall 2 — the ~5,000 character script limit. HeyGen caps input text per render. A 60–90 second reel script fits easily; a long-form script doesn't. Split long scripts into parts and stitch them, or keep client

deliverables short-form (you should anyway).

2 ElevenLabs — voice + timing

TOOL · THE INVISIBLE EDITOR

This is the tool nobody notices in the reel and the one the whole edit depends on. ElevenLabs **Scribe** is speech-to-text that returns a timestamp for **every single word** the avatar says. Once you know exactly when each word lands, everything an editor used to eyeball becomes math: captions light up on the spoken word, graphics pop at the name-drop, cuts land at sentence boundaries.

Send the rendered avatar video to the Scribe API (`scribe_v1` model), get back JSON like:

```
{ "text": "step one, render the avatar...",
  "words": [
    { "word": "step",   "start": 0.12, "end": 0.31 },
    { "word": "one",    "start": 0.35, "end": 0.58 },
    { "word": "render", "start": 0.90, "end": 1.24 }
  ] }
```

In my system it's one command, and the output feeds straight into the compositor:

```
# word-level timestamps for the rendered avatar video
python3.11 reel.py transcribe assets/avatar/avatar_video.mp4 words.json
```

Cost: about **$0.013 for a 2-minute reel**. One cent. That one cent replaces the sync work of a human editor — it's the best money in the entire stack.

ElevenLabs also covers the **voice** side when you need it: if a client's HeyGen voice clone sounds flat, clone their voice in ElevenLabs from the same filming-session audio and feed that audio into HeyGen instead. For faceless or B-roll content, their stock voices are strong enough to ship.

Why Scribe over free Whisper: I ran local Whisper first. It's free — and it mishears every proper noun ("Claude" → "clot", "n8n" → "n 8 n"), which means broken captions on exactly the words that matter in AI content. Scribe gets the product names right. For one cent per reel, this isn't a decision.

Pitfall 1 — proper nouns still deserve a scan. Even Scribe will trip on a brand-new tool name. Before compositing, grep the transcript for your key nouns and patch any misses — it's a 10-second check that saves a re-render.

Pitfall 2 — trim the loading frames first. Avatar renders often open with a few static frames before speech starts. Transcribe the *final trimmed* video, not the raw one — otherwise every timestamp is

offset and every caption fires early.

3 Claude (Claude Code) — the brain

TOOL · WRITES, PLANS, AND BUILDS — FROM ONE COMMAND

HeyGen is the face, ElevenLabs is the timing, but Claude Code is what makes this an **agency** instead of a hobby. It writes the scripts in each client's voice, decides how the video should be edited, and then actually runs the build — because the compositing layer is plain **ffmpeg**, which means every caption, graphic, zoom, and cut is a scriptable command Claude can execute. No timeline software. No render farm. A terminal.

The mechanism is a **skill**: a saved instruction file at `.claude/skills/<name>/SKILL.md` inside your project. Claude Code auto-discovers it and loads it whenever it's relevant — so the model already knows the client's voice, the tools, and the commands, every time, with zero re-explaining. One skill per job: a script writer, a video builder, a caption writer. Here's the starter skill to create first — paste this file in as-is:

.CLAUDE/SKILLS/SCRIPT-WRITER/SKILL.MD

```
---
name: script-writer
description: Write a 45-75 second talking-head reel script in the client's
  voice, ready to paste into a HeyGen render. Use whenever the user gives a
  topic or says "write a reel about".
---
You write short-form video scripts for [CLIENT NAME], [one line: who they are
and who they serve]. Voice: confident, direct, friend-to-friend. Never corporate.

VOICE SAMPLE (match this exactly — rhythm, word choice, sentence length):
[paste 3 of the client's real captions, emails, or a talk transcript here]

RULES
1. Hook in the first 3 seconds — open a loop, never close it in the hook.
2. One core idea per script. Two ideas = two scripts.
3. Spoken 8th-grade English. Short sentences. Active voice. No filler intro.
4. Name at least ONE specific tool, number, or timeframe (that's the save trigger).
5. Build to one quotable line near the end — the screenshot line.
6. End with a comment-keyword CTA: "Comment [KEYWORD] and I'll send you [thing]."
7. Length: 110-170 words (45-75 seconds spoken). NEVER exceed 5,000 characters —
   that's the HeyGen input limit.

OUTPUT: the spoken script only, as plain text, ready for the render.
```

Then the workflow is a conversation:

PROMPT — WRITE THE SCRIPT

Use the script-writer skill. Client: [name]. Topic: "why founders waste 10 hours a week on content." Keyword CTA: SYSTEM. Return the script as plain text I can save to script.txt.

And the build is three commands Claude runs for you (this is my real production line):

```
# 1. render the avatar on a look from the client's library
python3.11 reel.py render green_bookshelf_front --script-file script.txt --yes

# 2. word-level timestamps via Scribe
python3.11 reel.py transcribe assets/avatar/avatar_video.mp4 words.json

# 3. composite captions + graphics, auto-finish (speed pass + thumbnail end-card)
python3.11 reel.py make authority_stack --avatar assets/avatar/avatar_video.mp4 --
captions words.json
```

Under the hood, step 3 is ffmpeg overlays driven by the Scribe timestamps — captions appear at `word_start`, graphics pop a fraction before the word that names them. Because it's all code, Claude can build it, check it, and rebuild it without you opening an editor once.

The 1.3× trick: avatar renders speak at ~140 words/min, which reads sluggish on a feed. The finish pass speeds the final video to ~185 wpm — the pace top creators actually post at. One ffmpeg flag, massive retention difference. Bake it into the pipeline so you never forget it.

Pitfall 1 — prompts don't compound, skills do. If you re-explain the client's voice in every chat, you're doing the work the SKILL.md should do once. One skill file per client, with their real writing pasted into the voice sample block.

Pitfall 2 — lock the script before you render. Script rewrites are free; avatar renders cost credits. Get the script approved (by you or the client) first, render second. Never render a draft.

---

4 Running it for clients

THE AGENCY MODEL · FILM ONCE, DELIVER FOREVER

Here's how the three tools become a service:

- **Onboarding = one filming session.** Book a videographer for a half-day with the client. Capture 4–6 looks: 2–3 outfits, 2 settings, front + side angles, ~2 minutes of natural talking each, plus the voice-consent line. That session is the only time the client ever films.
- **Train the library that week.** Upload every clip, train every look, verify each one's framing with a test render. Build their `script-writer` skill from their real captions and emails.
- **Then run a weekly calendar.** You propose topics, Claude drafts the scripts, the client approves them in a shared doc (approving text takes them 5 minutes — this is the entire client workload), and you render + composite + deliver. Rotate the looks across the week so the feed never looks like one recording session.
- **How I structure pricing:** a one-time **setup fee** that covers the filming session, avatar training, and voice/skill setup — then a **monthly retainer** for the ongoing videos. The setup fee anchors the value of the library (it's a real asset the client keeps); the retainer is where the business compounds, because your marginal cost per video is credits and cents, not hours.
- **The pitch writes itself:** "You film once. You approve scripts from your phone. You post daily without ever booking a studio again." Show them a sample render of *themselves* from the trial session — that demo closes more deals than any deck.

Scale math: the marginal cost of one client video is a few HeyGen credits + one cent of Scribe + zero editing hours. Every new client adds a filming day and a skill file — not headcount. That's why this works as a one-person agency.

Quickstart checklist — your first client (even if it's you)

1. Film one session: 4–6 looks, ~2 min each, natural gestures, plus the voice-consent line.
2. Train each look in HeyGen; name them `outfit_setting_angle`; test-render each and verify framing.
3. Get an ElevenLabs API key; run one Scribe transcription on a test render and open the words JSON.
4. Install Claude Code; create `.claude/skills/script-writer/SKILL.md` from the block above with real voice samples pasted in.
5. Produce one full video end-to-end: script → approve → render → transcribe → composite → 1.3× finish.
6. Post it. Then book the calendar: one video per week minimum, looks rotated.

The 5 mistakes that stall beginners

- **Training one look and calling it done** — a single set on repeat reads as AI within a week. The library (outfits × settings × angles) is what makes the feed feel alive.
- **Skipping the framing check on a new look** — every setting puts the face somewhere different. One un-verified look = one delivered video with a cropped head.
- **Eyeballing caption sync instead of using Scribe** — hand-timed captions are why edits take hours. One cent of word timestamps replaces the entire sync job.
- **Rendering unapproved scripts** — text is free, credits aren't. The approval step isn't bureaucracy; it's your margin.
- **Charging per video instead of setup + retainer** — per-video pricing sells your hours. Setup + retainer sells the system, and the system is the product.

This is the exact system I run at ZeroHands

Three tools, one filming session per client, video every week. I share the real builds — screens, prompts, and the misses too — one automation at a time.

Follow **@automatewithanand** on Instagram · Subscribe on YouTube · Comment **AGENCY** on the reel if you want the next guide first.